

Federated & Swarm Learning

Thoai Nam

High Performance Computing Lab (HPC Lab)

Faculty of Computer Science and Technology

HCMC University of Technology

Distributed ML/Deep Learning

ML: high level view

- Notation
 - D : data
 - A : model parameters
 - L : function to optimize (e.g., minimize loss)
- Goal: Update A based on D to optimize L
- Typical approach: iterative convergence

The diagram illustrates the iterative update of model parameters A . The equation $A^t = F(A^{(t-1)}, \Delta_L(A^{(t-1)}, D))$ is shown. A blue arrow points from the text "iteration t " to A^t . Another blue arrow points from the text "compute updates that minimize L " to $\Delta_L(A^{(t-1)}, D)$. A curved blue arrow points from the text "merge updates to parameters" to the function F .

$$A^t = F(A^{(t-1)}, \Delta_L(A^{(t-1)}, D))$$

iteration t *compute updates that minimize L* *merge updates to parameters*

Abstracting ML algorithms

- Can we find commonalities among ML algorithms?
- This would allow finding
 - Common abstractions
 - Systems solutions to efficiently implement these abstractions
- Some common aspects
 - We have a prediction model A
 - A should optimize some complex *objective function* L
 - ML algorithm does this by iteratively refining A

High level view

- Notation
 - D : data
 - A : model parameters
 - L : function to optimize (e.g., minimize loss)
- Goal: Update A based on D to optimize L
- Typical approach: iterative convergence

$$A^t = F(A^{(t-1)}, \Delta_L(A^{(t-1)}, D))$$

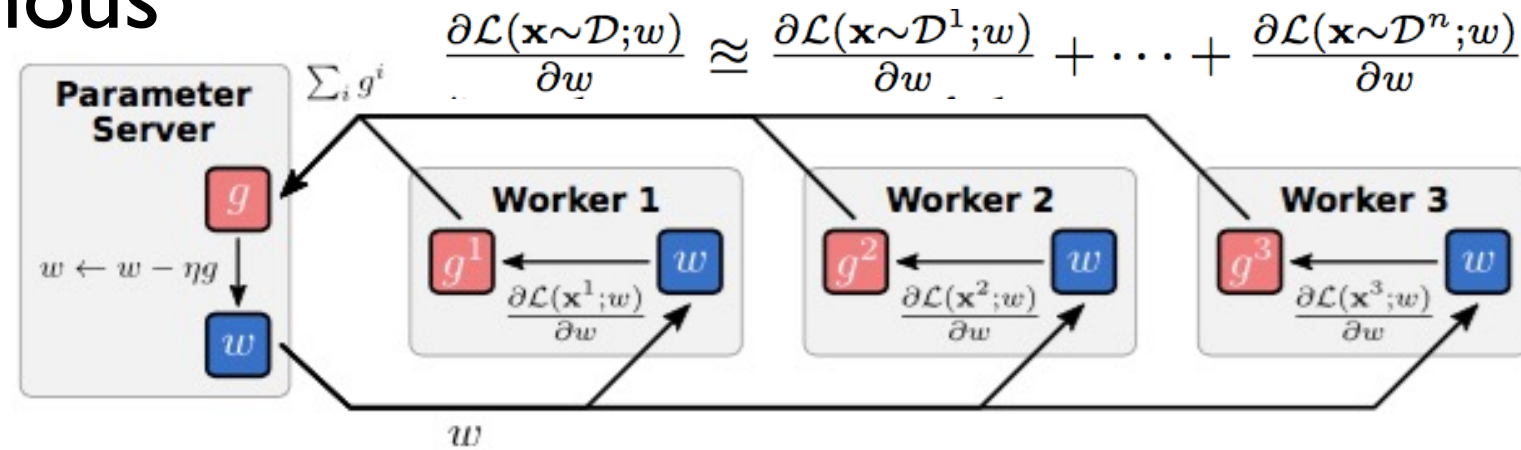
iteration t

compute updates that minimize L

merge updates to parameters

Centralized Synchronous systems

- Each training cycle begins with the workers *downloading* new model parameters (w) from the parameter server
- Workers locally sample a training mini- batch ($x \sim \mathcal{D}^i$) and *compute* per-parameter gradients (g^i)
- Workers *share* their gradients with the parameter server
- The parameter server *aggregates* the gradients from all workers and injects the aggregate into an optimization algorithm to *update* the model.



PARAMETER SERVER PROGRAM

Require: initial model $\tilde{w}_0$, learning rate η , number of workers n

```

1: for  $t \leftarrow 0, 1, 2, \dots$  do
2:   Broadcast  $\tilde{w}_t$ 
3:   Await gradients  $g_t^i$  from all workers
4:    $\tilde{w}_{t+1} \leftarrow \tilde{w}_t - \eta \sum_{i=1}^n g_t^i$ 
5: end for

```

PROGRAM OF THE i^{th} WORKER

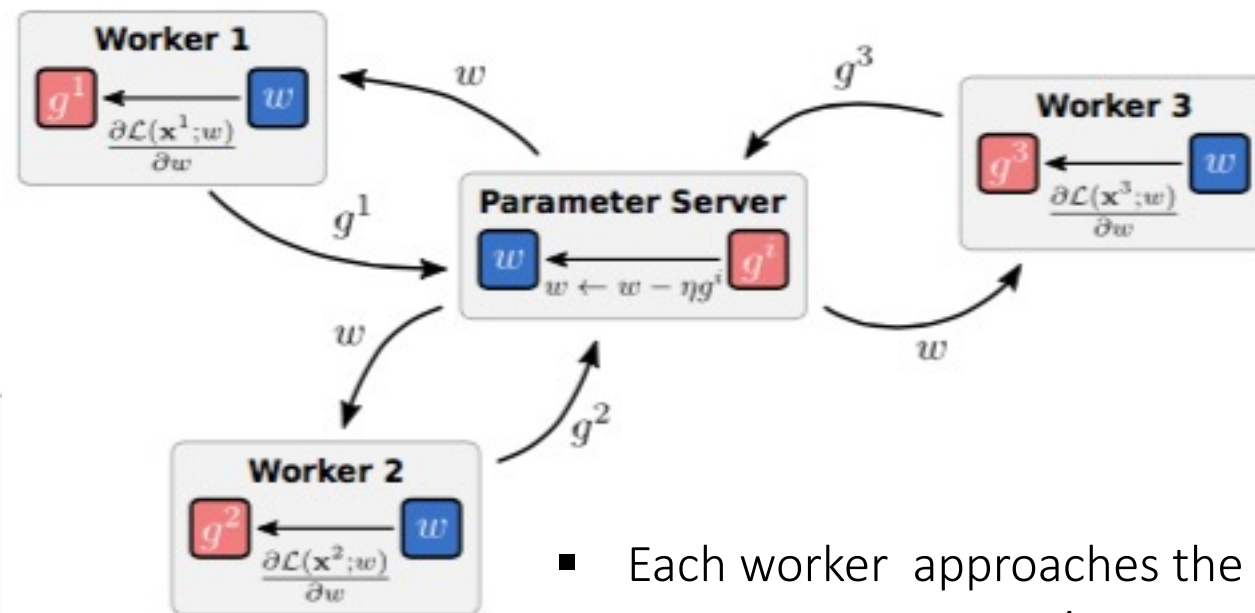
Require: training data source $\mathcal{D}^i$

```

1: for  $t \leftarrow 0, 1, 2, \dots$  do
2:   Await  $\tilde{w}_t$ 
3:   Sample mini-batch  $\mathbf{x} \sim \mathcal{D}^i$ 
4:    $g_t^i \leftarrow \frac{\partial \mathcal{L}(\mathbf{x}; \tilde{w}_t)}{\partial \tilde{w}_t}$ 
5:   Send  $g_t^i$  to parameter server
6: end for

```

Centralized Asynchronous systems



PARAMETER SERVER PROGRAM

Require: initial model state $\tilde{w}_0$, learning rate η

```

1:  $\tilde{w} \leftarrow \tilde{w}_0$ 
2: Distribute  $\tilde{w}$ 
3: for  $t \leftarrow 0, 1, 2, \dots$  do
4:   if received gradients  $(g_{t-\delta}^i)$  from worker  $i$ , with a delay of  $\delta$  steps then
5:      $\tilde{w} \leftarrow \tilde{w} - \eta g_{t-\delta}^i$ 
6:     Send  $\tilde{w}$  to worker  $i$ 
7:   end if
8: end for

```

PROGRAM OF THE i^{th} WORKER

Require: training data source $\mathcal{D}^i$

```

1: for  $t^i \leftarrow 0, 1, 2, \dots$  do
2:   Await of current parameter server model  $\tilde{w}$ 
3:    $w^i \leftarrow \tilde{w}$ 
4:   Sample mini-batch  $\mathbf{x} \sim \mathcal{D}^i$ 
5:    $g^i \leftarrow \frac{\partial \mathcal{L}(\mathbf{x}; w^i)}{\partial w^i}$ 
6:   Send  $g^i$  to parameter server
7: end for

```

- Each worker approaches the parameter server at its own pace to offer gradient, after which the global model is updated immediately, and request updated model parameters
- Each worker maintains a separate parameter exchange cycle with the parameter server
- There is no interdependence between workers, situations where straggler nodes delay the execution of other workers cannot happen.

Federated Learning

A shift of paradigm: from centralized to decentralized data

- The standard setting in Machine Learning (ML) considers a centralized dataset processed in a tightly integrated system
- But in the real world data is often decentralized across many parties.



Decentralized data

- Sending the data may be too costly
 - Self-driving cars are expected to generate several TBs of data a day
 - Some wireless devices have limited bandwidth/power
- Data may be considered too sensitive
 - We see a growing public awareness and regulations on data privacy
 - Keeping control of data can give a competitive advantage in business and research.

How about each party learn on its own

- The local dataset may be too small
 - Sub-par predictive performance (e.g., due to overfitting)
 - Non-statistically significant results (e.g., medical studies)
- The local dataset may be biased
 - Not representative of the target distribution.



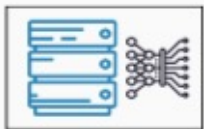
A broad definition of federated learning

- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized.



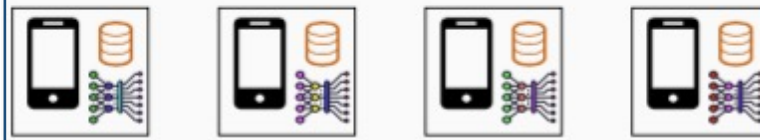
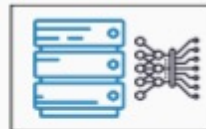
Initialize model

1



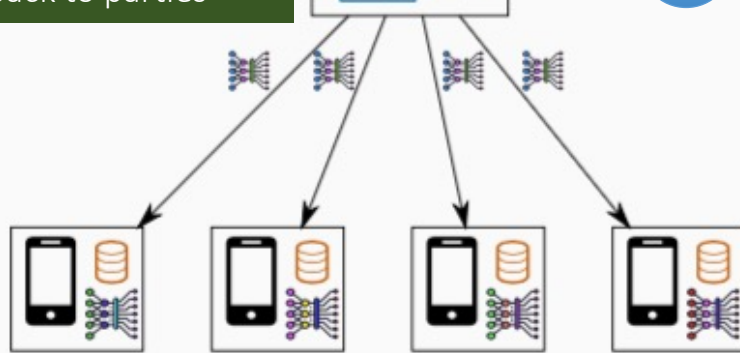
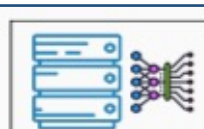
Each party make
an update using its
local data set

2



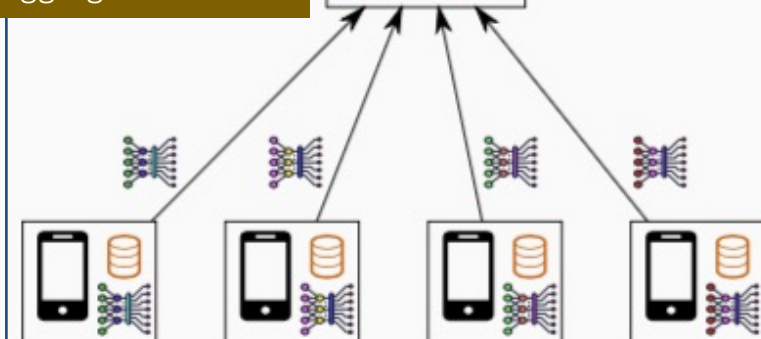
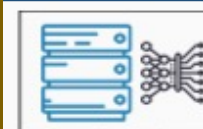
Server aggregates
updates and sends
back to parties

4



Parties share local
updates for
aggregation

3



Iterative

A broad definition of federated learning

- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized
- We would like the final model to be as good as the centralized solution (ideally), or at least better than what each party can learn on its own.

Key differences with distributed learning

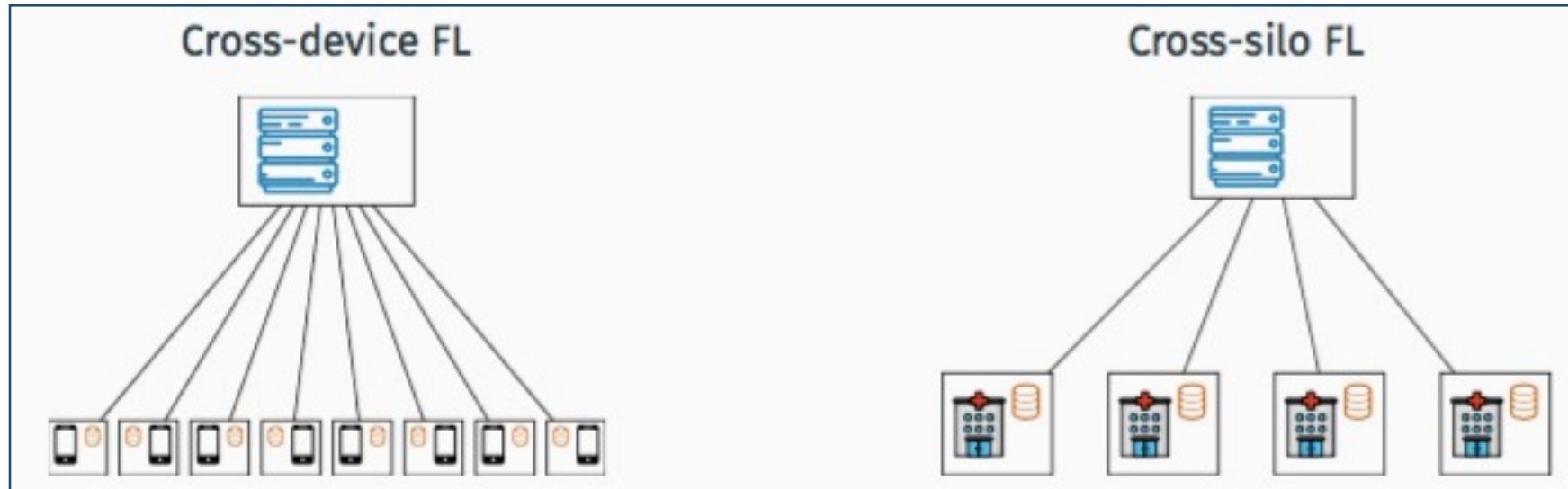
■ Data distribution

- In distributed learning, data is centrally stored (e.g., in a data center)
 - ✧ The main goal is just to **train faster**
 - ✧ We control how data is distributed across workers: usually, it is **distributed uniformly at random across workers**
- In FL, data is naturally distributed and generated locally
 - ✧ Data is not independent and identically distributed (**non-i.i.d.**), and it is **imbalanced**

■ Additional challenges that arise in FL

- Enforcing privacy constraints
- Dealing with the possibly limited reliability/availability of participants
- Achieving robustness against malicious parties
- ...

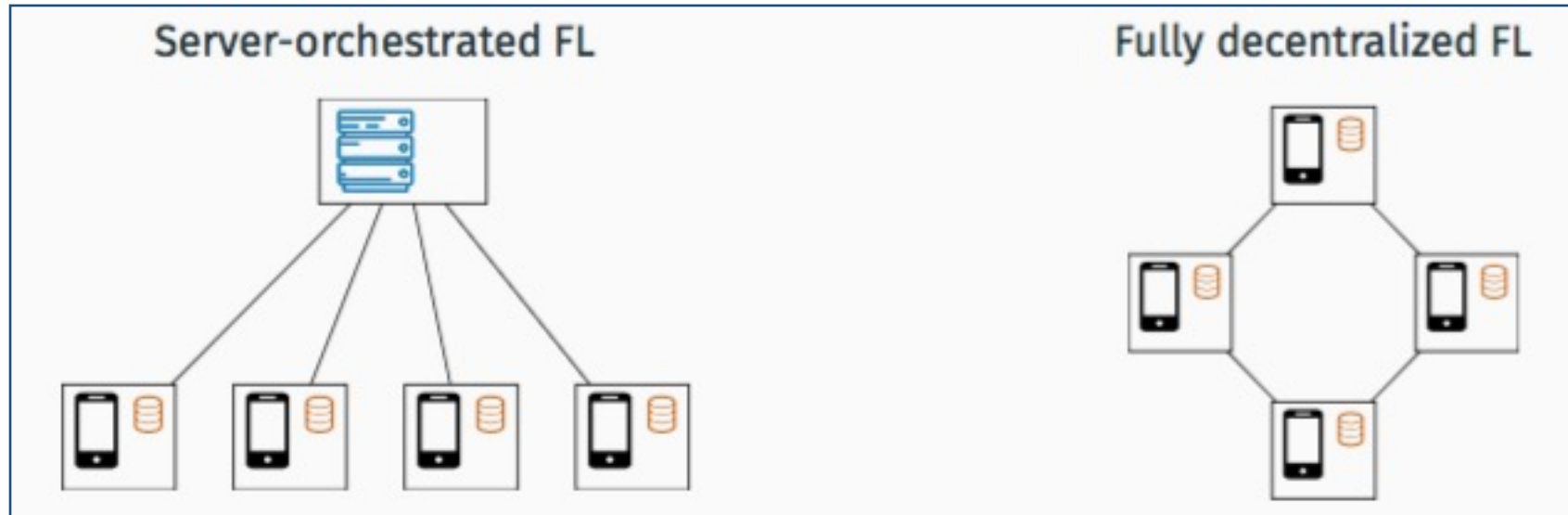
Cross-device vs. cross-silo FL



- Massive number of parties (up to 10^{10})
- Small dataset per party (could be size 1)
- Limited availability and reliability
- Some parties may be malicious.

- 2-100 parties
- Medium to large dataset per party
- Reliable parties, almost always available
- Parties are typically honest.

Server orchestrated vs. fully decentralized FL



- Server-client communication
- Global coordination, global aggregation
- Server is a single point of failure and may become a bottleneck.

- Device-to-device communication
- No global coordination, local aggregation
- Naturally scales to a large number of devices.

A new paradigm – Federated Learning

- *A synchronous update scheme that proceeds in rounds of communication*
- For efficiency, at the beginning of each round, *a random fraction C of clients* is selected, and the server sends the current model parameters to each of these clients.

[McMahan, H. Brendan, Eider Moore, Daniel Ramage, and Seth Hampson. "Communication-efficient learning of deep networks from decentralized data." *AISTATS*, 2017.]

Federated Learning

- Recall in traditional deep learning model training
 - For a training dataset containing n samples (x_i, y_i) , $1 \leq i \leq n$, the training objective is:

$$\min_{w \in \mathbb{R}^d} f(w) \quad \text{where} \quad f(w) \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n f_i(w)$$

$f_i(w) = l(x_i, y_i, w)$ is the loss of the prediction on example (x_i, y_i)

- Deep learning optimization relies on SGD and its variants, through *mini-batches*

$$w_{t+1} \leftarrow w_t - \eta \nabla f(w_t; x_k, y_k)$$

Federated Learning

- In federated learning

- Suppose n training samples are distributed to K clients, where P_k is the set of indices of data points on client k , and $n_k = |P_k|$.

- For training objective: $\min_{w \in \mathbb{R}^d} f(w)$

$$f(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w) \quad \text{where} \quad F_k(w) \stackrel{\text{def}}{=} \frac{1}{n_k} \sum_{i \in P_k} f_i(w)$$

A baseline – *FederatedSGD (FedSGD)*

- A randomly selected client that has n_k training data samples in federated learning
≈ A randomly selected sample in traditional deep learning
- Federated SGD (FedSGD): a single step of gradient descent is done per round
- Recall in federated learning, a C -fraction of clients are selected at each round
 - $C = 1$: full-batch (non-stochastic) gradient descent
 - $C < 1$: stochastic gradient descent (SGD).

Federated learning – *FederatedAveraging (FedAvg)*

- Learning rate: η ; total #samples: n ; total #clients: K ; #samples on a client k : n_k ; clients fraction $C = 1$
- In a round t :
 - The central server broadcasts current model w_t to each client; each client k computes gradient: $g_k = \nabla F_k(w_t)$, on its local data
 - Approach 1: Each client k submits g_k ; the central server aggregates the gradients to generate a new model:

$$w_{t+1} \leftarrow w_t - \eta \nabla f(w_t) = w_t - \eta \sum_{k=1}^K \frac{n_k}{n} g_k.$$

$$\text{Recall } f(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w)$$

- Approach 2: Each client k computes: $w_{t+1}^k \leftarrow w_t - \eta g_k$; the central server performs aggregation:

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

For multiple times $\Rightarrow$ FederatedAveraging (FedAvg)

Federated learning – deal with limited communication

- Increase computation
 - Select more clients for training between each communication round
 - Increase computation on each client.

A baseline – *FederatedSGD (FedSGD)*: *computation cost*

➤ Learning rate: η ; total #samples: n ; total #clients: K ; #samples on a client k : n_k ; *clients fraction C*

■ In a round t :

○ The central server broadcasts current model w_t to each client; each client k computes gradient: $g_k = \nabla F_k(w_t)$, on its local data.

➤ Approach 2:

- Each client k computes for E epochs: $w_{t+1}^k \leftarrow w_t - \eta g_k$
- The central server performs aggregation: $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$
- Suppose B is the local mini-batch size, #updates on client k in each round: $u_k = E \frac{n_k}{B}$.

A baseline – *FederatedSGD (FedSGD)*

Algorithm 1 FederatedAveraging. The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

initialize w_0

for each round $t = 1, 2, \dots$ do

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$ (random set of m clients)

for each client $k \in S_t$ in parallel do

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$

ClientUpdate(k, w): // Run on client k

$\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)

for each local epoch i from 1 to E do

for batch $b \in \mathcal{B}$ do

$w \leftarrow w - \eta \nabla \ell(w; b)$

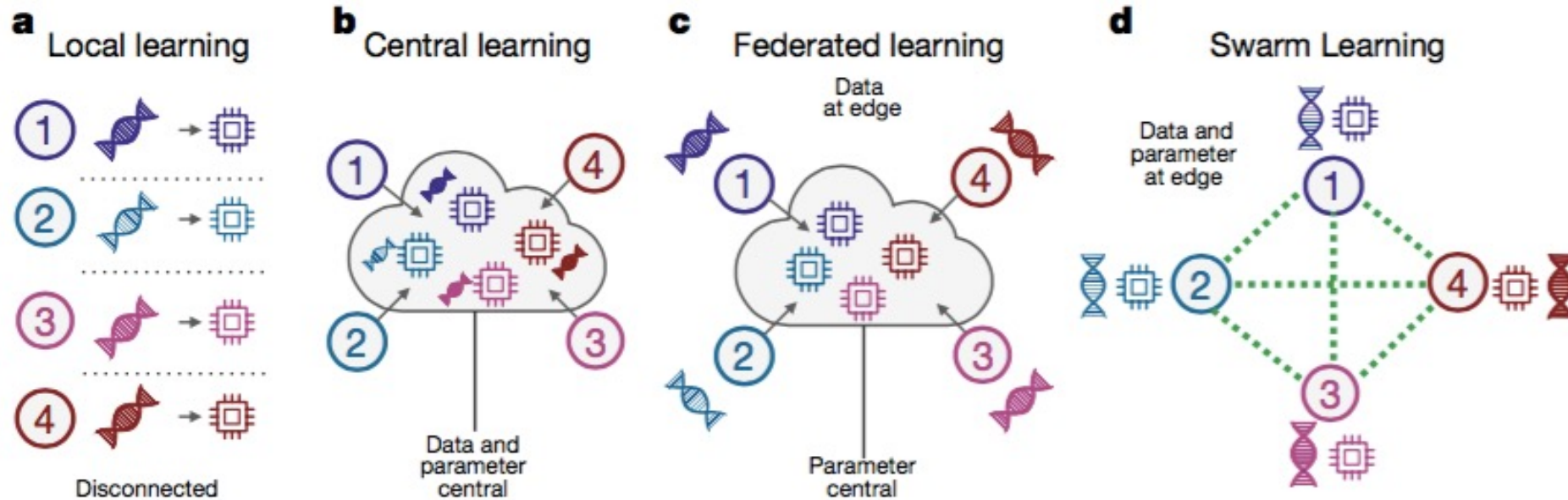
return w to server

1. At first, a model is randomly initialized on the central server.
2. For each round t :
 - (a) A random set of clients are chosen;
 - (b) Each client performs local gradient descent steps;
 - (c) The server aggregates model parameters submitted by the clients.

Swarm Learning

1. Warnat-Herresthal, S., Schultze, H., Shastry, K.L. et al. Swarm Learning for decentralized and confidential clinical machine learning. Nature 594, 265–270 (2021). <https://doi.org/10.1038/s41586-021-03583-3>.
2. Jialiang Han, Yun Ma, Yudong Han:Demystifying Swarm Learning: A New Paradigm of Blockchain-based Decentralized Federated Learning. CoRR abs/2201.05286 (2022).
3. <https://github.com/spiffe/spire>.
4. <https://spiffe.io/docs/latest/spiffe-about/overview/>.

Swarm learning



(a) illustration of the concept of local learning with data and computation at different, disconnected locations.

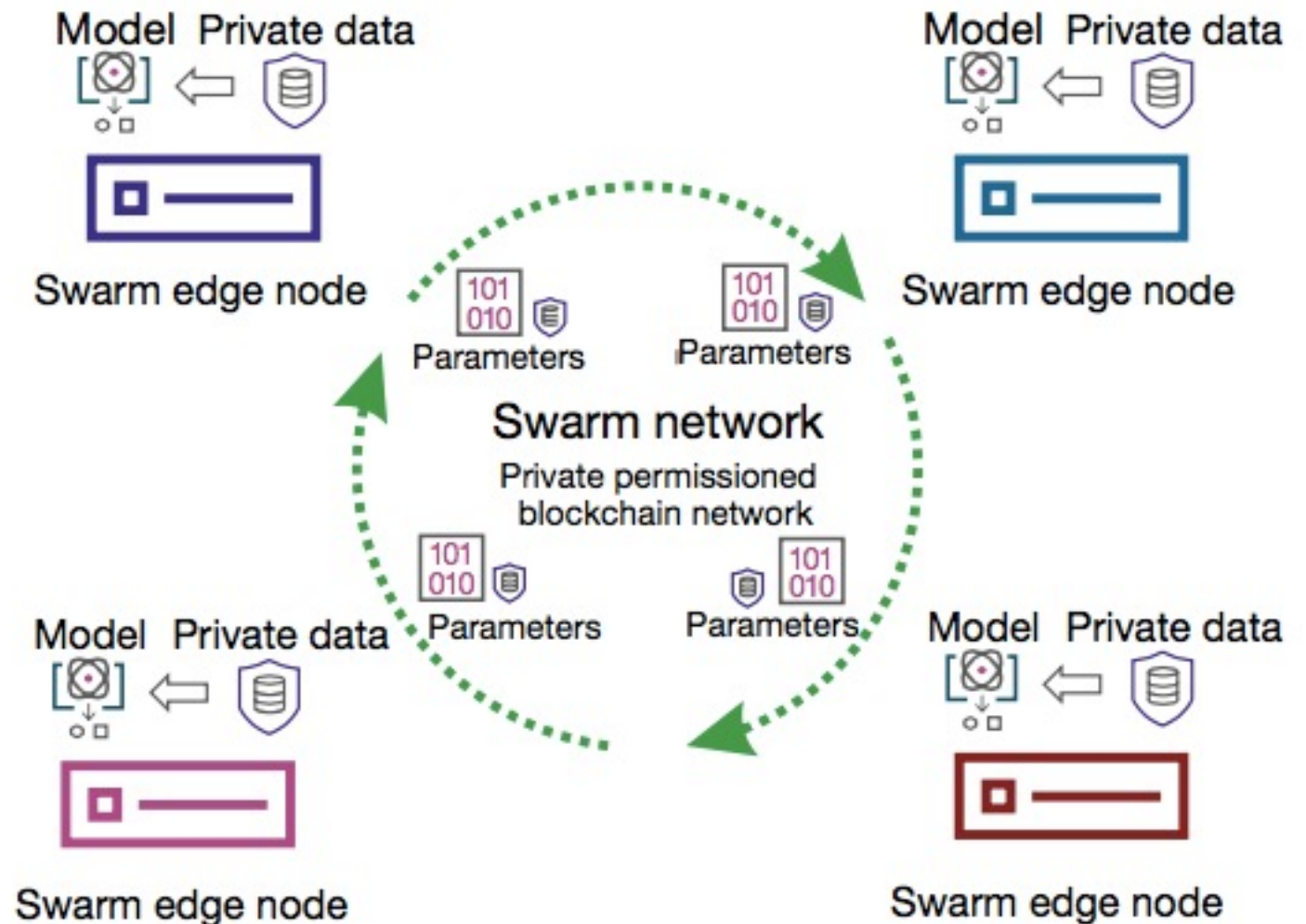
(b) In cloud computing, data are moved centrally so that machine learning can be carried out by centralized computing.

(c) Federated learning, with data being kept with the data contributor and computing performed at the site of local data storage and availability, but parameter settings orchestrated by a central parameter server

(d) Swarm learning, which dispenses with a dedicated server, shares the parameters via the Swarm network and builds the models independently on private data at the individual sites. however, a remainder of a central structure is kept.

Schematic of the Swarm network

- SL shares the parameters via the Swarm network and builds the models independently on private data at Swarm edge nodes, without the need of a central custodian
- With the benefit of blockchain, SL secures data sovereignty, security, and confidentiality
- Each participant is well defined and only pre-authorized participants can be onboarded and execute transactions
- Private data are used at each node together with the model provided by the Swarm network.

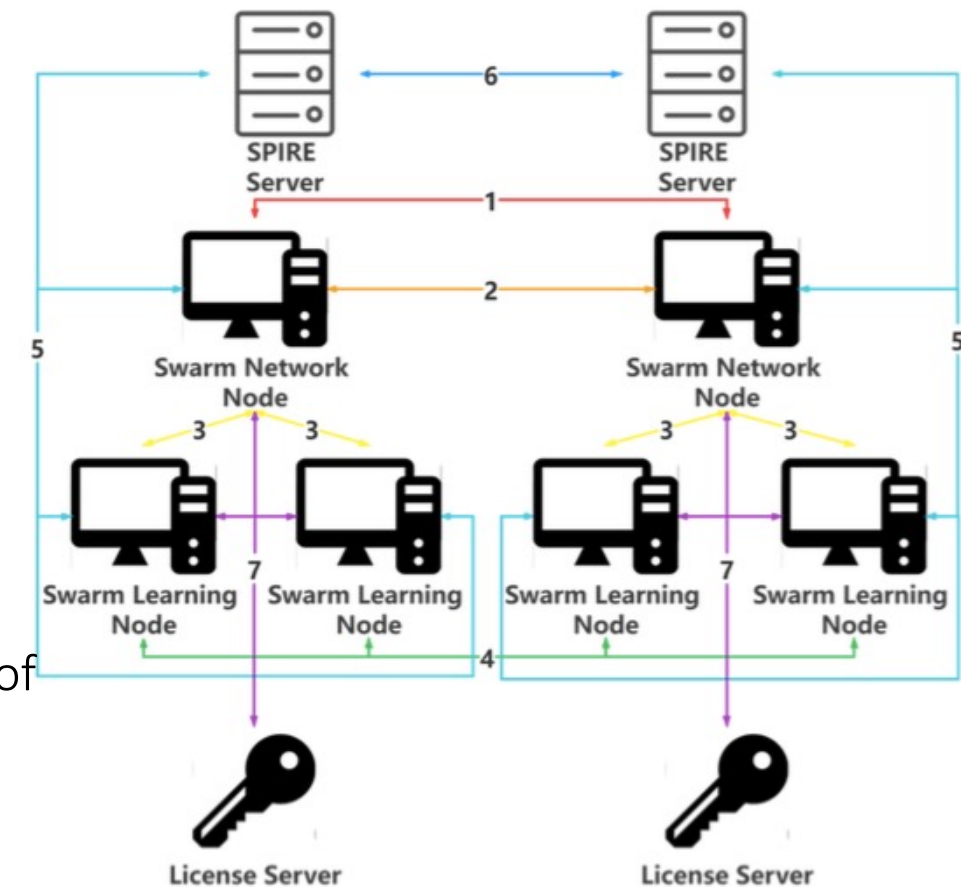


The workflow of SL

- A new edge node enrolls via a blockchain smart contract, obtains the model, and performs localized training until a user-defined Synchronization Interval (SI)
- Local model parameters are exchanged and merged to update the global model before the next training round
- Apart from Swarm edge nodes, there are Swarm coordinator nodes responsible for maintaining metadata like the model state, training progress, and licenses, without model parameters.

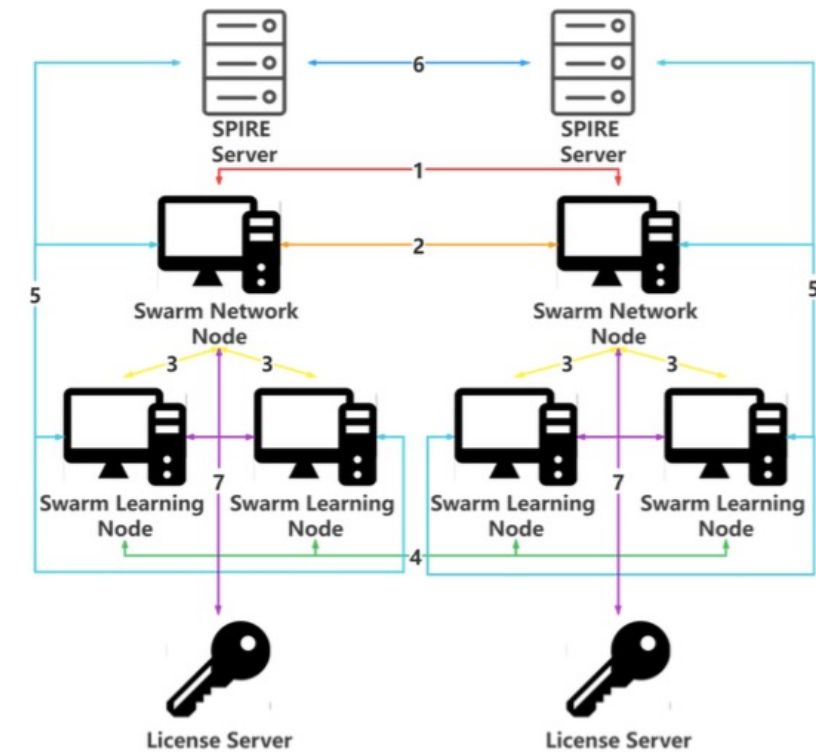
SL framework (I)

- **Swarm Learning (SL) node** runs a user-defined DL algorithm, which iteratively updates the local model
 - **Swarm Network (SN) node** interacts with each other to maintain global state information about the model and tracks training progress via the Ethereum blockchain platform
 - ✧ Besides, each SL node registers itself with an SN node when initialization and each SN node coordinates the training pipeline of its SL nodes
 - ✧ Note that the blockchain records only metadata like the model state and training progress, without model parameters
 - **Swarm Learning Command Interface (SWCI) node** securely connects to SN nodes to view the status, control, and manage the SL framework
 - **SPIFFE SPIRE Server node** guarantees the security of the SL framework
- Each SN node or SL node includes a SPIRE Agent Workload Attestor plugin that communicates with the SPIRE Server nodes to attest their identities and to acquire and manage a SPIFFE Verifiable Identity Document (SVID)
- **License Server (LS) node** installs and manages the license to run the SL framework.



SL framework (2)

- **Swarm Network Peer-to-Peer Port** is used by SN nodes to share state information about the blockchain with each other, i.e. Line 1
- **Swarm Network File Server Port** is used by SN nodes to run file servers and share state information about the SL framework with each other, i.e. Line 2
- **Swarm Network API Port** is used by SN nodes to run REST-based API servers, i.e. Line 3. The API server is used by SL nodes to send and receive state information from the SN node they are registered with. It is also used by SWCI nodes to view and manage the status of the SL framework
- **Swarm Learning File Server Port** is used by SL nodes to run file servers and share learned parameters of the model with each other, i.e. Line 4
- **SPIRE Server API Port** is used by SPIRE Server nodes to run gRPC-based API servers for SN nodes and SL nodes to connect to the SPIRE Server and acquire SVIDs, i.e. Line 5
- **SPIRE Server Federation Port** is used by SPIRE Server nodes to connect each other in the SPIRE federation and send and receive trust bundles, i.e. Line 6
- **License Server API Port** is used by the LS node to run a REST-based API server and a management interface, i.e. Line 7. The API server is used by SN nodes and SL nodes to connect to the LS node and acquire licenses. The management interface is used by the SL framework administrators to connect to the LS node from browsers and administer licenses.



The workflow of SL node

An SL node initializes in the following pipeline

- ① It acquires a license from the LS node
 - ② It acquires an SVID from the SPIRE Server node
 - ③ It registers itself with an SN node
 - ④ It starts a file server and announces to the SN node that it is ready to run the training program
 - ⑤ It starts running the user-specified model training program.
- After initialization, each SL node regularly shares its learned parameters with other SL nodes and merges their parameters
 - This merging periodicity is defined by a Synchronization Interval (SI), which specifies the number of training batches after which SL nodes merge their learned parameters
 - At the end of each SI, one of the SL nodes is elected as the leader by the blockchain
 - The leader collects local models from each SL node and merges them into a global model by averaging their learned parameters
 - After that, each SL node receives this merged model and starts the next SI.